Assignment 3: Plan the Campus Festival

Imagine that your team is preparing a campus festival. The entrance, stage, food area and workshops need cables, a preparation schedule and delivery routes. You have three different kinds of information: cables connect two places in both directions; a preparation dependency goes from one task to another; a delivery route also has a direction and a cost.

Your task is to complete a small program that answers five questions about these graphs. The program framework is already in the GitHub template. You will write the algorithms that are currently marked TODO, test them and explain how they work. Choose **one** language: Java, Python, C++ or C#. You do not need a website, interface, database or a separate festival simulation.

The festival is the setting for the problems. In the program, places and preparation tasks are represented by numbered vertices: 0, 1, 2 and so on. An edge is a connection between two vertices. Each part uses its own graph; the cable network and the preparation schedule are not the same input.

Your roadmap

1. Make your own copy of the GitHub template and choose your language. Open the Algorithms file listed below. The input/output code is already provided.

2. Complete DSU: keep track of connected groups as new cables are added.

3. Implement both Kruskal and Prim: choose a cable network with the smallest total cost.

4. Implement Kosaraju: find groups of preparation tasks with circular dependencies.

5. Implement DFS topological ordering: decide whether the preparation tasks can be scheduled and return an order.

6. Implement shortest paths in a DAG: find the cheapest delivery routes and reconstruct a route.

7. Run the ready-made tests in GitHub, add your own examples and complete the short explanation in ANALYSIS.md. Then submit the exact commit link and its successful test-run link in Moodle.

Where to write your code

Language	File with the algorithms
Java	Java/source/Algorithms.java
Python	Python/source/algorithms.py
C++	Cpp/source/algorithms.hpp
C#	CSharp/source/Algorithms.cs

Fill in the six methods dsu, kruskal, prim, scc, topo and dag in your selected file. You may add helper functions/classes and additional source files directly in the same source folder. Keep the supplied main program and JSON input/output working. You only implement your chosen language; the other folders are alternatives.

Part 1. Which festival places are connected?

A cable is added between two places. If they were in different groups, those groups become one. If they were already connected, nothing changes. After every cable, report the number of connected groups and the size of the largest group.

Write dsu using a disjoint-set union structure. Implement find and union yourself, with **path compression** and **union by size or rank**. At the beginning, every vertex is a separate group.

Example: there are four places, numbered 0–3. The cables arrive in this order:

New cable	Groups remaining	Largest group
0–1	3	2
1–2	2	3
0–2	2	3
3–3	2	3

The third cable does not merge anything: 0 and 2 are already connected through 1. A cable from 3 to itself also changes nothing. Your states result is [[3,2],[2,3],[2,3],[2,3]].

Input fields: n is the number of vertices; operations is the list of vertex pairs. Output: states, containing [number of groups, largest size] after each operation. Repeated pairs and self-unions are valid. For n=0, there are no operations and the result is an empty list; the largest group size is 0.

Target complexity: O((n+q) alpha(n)) amortized time for q operations, with O(n+q) memory including the result.

Part 2. Which cables should we buy?

Now every possible cable has a cost. Choose cables that connect every place that can be connected, without cycles, and minimise their total cost. If the graph is disconnected, build a separate tree for each connected group. Together these trees form a **minimum spanning forest**. An isolated vertex remains a component too.

Solve this problem twice, with separate implementations:

1. kruskal: sort the edges by weight and use DSU to avoid adding a cycle.

2. prim: use adjacency lists and a priority queue to expand one component; restart at an unvisited vertex when necessary.

Return cost, edge_ids and components. An edge ID is its position in the input edges list, starting at 0. Each chosen ID must occur once. Do not choose self-loops. Parallel edges and negative costs are allowed. Several forests may have the same minimum cost: any valid optimum is accepted.

Example: n=4, with edges [0,1,4], [1,2,2], [0,2,3]. Choose edge IDs 1 and 2. Their total cost is 5. Vertex 3 is isolated, so components=2. Both algorithms must find cost 5, although on inputs with ties their chosen IDs may differ.

Input: n and edges, where each edge is [u,v,weight]. Output: {cost,edge_ids,components}. For n=0, return cost=0, edge_ids=[] and components=0.

Compare Kruskal and Prim on the **same graphs at three input sizes**. Record the sizes, how the graphs were produced, run commands or Actions links, three or more repeated measurements and median times in ANALYSIS.md. Explain the difference you observe; this is a comparison of algorithms, not a speed competition between students.

Target complexity: Kruskal O(n+m log(m+1)); Prim O(n+m log(m+1)) with a lazy heap, or O((n+m) log(n+1)) with an indexed heap. Memory: O(n+m).

Part 3. Which preparation tasks depend on one another?

An edge u→v means that u must be completed before v. Sometimes dependencies form a loop: for example, 0 requires 1 while 1 also requires 0. Find the **strongly connected components**: maximal groups in which every vertex can reach every other vertex following directed edges.

Write scc using **Kosaraju**, with two DFS passes and a transposed graph. Give each vertex a component label. Labels may be any nonnegative integers, but equal labels must mean exactly one SCC.

Also return the condensation graph: replace every SCC by one vertex, then include each directed connection between different components once. Remove connections inside a component and repeated component pairs.

Example: edges 0→1, 1→0, 1→2, 0→2, 2→3. One valid answer has component labels [5,5,7,9] and condensation edges [[5,7],[7,9]]. Vertices 0 and 1 form one SCC; the two links from that SCC to vertex 2 become one link.

Input: n and edges of the form [u,v]. Output: component, with one label per vertex, and condensation, with pairs of labels. Isolated vertices have their own components. Loops and duplicate edges are allowed. For n=0, both arrays are empty. The condensation is a DAG, but finding an SCC does not remove the original scheduling conflict inside it.

Target complexity: O(n+m) expected time with a hash set for removing duplicate pairs; O(n+m) memory. Use a DFS implementation that handles deep graphs without a recursion-stack failure.

Part 4. In what order should we prepare?

Using a directed dependency graph, return a sequence in which every task appears once and every prerequisite appears before the task that depends on it. This is a **topological order**. If any directed cycle exists, such an order is impossible.

Write topo with DFS, using three states: not visited, active and finished. An edge to an active vertex reveals a cycle. For an acyclic graph, reverse the DFS finishing order. Do not replace this required method with Kahn's algorithm.

Example: dependencies 0→2, 1→2, 2→3 allow [1,0,2,3] or [0,1,2,3]. Both 0 and 1 must come before 2, and 2 must come before 3.

Input: n and edges [u,v]. Output: acyclic and order. If a cycle exists anywhere, return acyclic=false and order=[]. Otherwise return acyclic=true and any valid order containing all vertices. For an empty graph, return true and []. Include isolated vertices and handle loops, duplicate edges and deep graphs.

Target complexity: O(n+m) time and memory, including the DFS stack.

Part 5. What is the cheapest delivery route?

A directed delivery graph has a cost on each edge. From a given source, find the cheapest route to every vertex. This part uses a **DAG**: a directed graph without cycles. Negative edge costs are allowed; for example, an edge may represent a saving.

Write dag. First check that the **whole graph** is acyclic. A cycle in an unreachable area still makes the input unsuitable: return acyclic=false, distance=[] and parent_edge=[]. Otherwise process vertices in topological order and relax outgoing edges. Never relax from an unreachable vertex.

Return distance, with the minimum cost for every vertex, and parent_edge, with the incoming edge ID chosen for that shortest route. At the source, distance=0 and parent_edge=-1. At an unreachable vertex, distance=null and parent_edge=-1. Other parent edges must form valid shortest paths from the source. Any shortest path is accepted when costs tie.

Example: source=0, n=5, edges [0,1,3], [0,2,8], [1,2,-7], [2,3,4]. The distances are [0,3,-4,0,null], and one parent_edge array is [-1,0,2,3,-1]. The route to 3 is 0→1→2→3, with cost 3−7+4=0. Vertex 4 cannot be reached.

Add a helper that reconstructs the vertex route from parent_edge. Show its use on a reachable vertex, the source itself and an unreachable vertex in your own tests or report. A minimum spanning tree from Part 2 does not solve this shortest-path problem.

Input: n, edges [u,v,weight] and source. Output: {acyclic,distance,parent_edge}. Parallel edges are allowed; a self-loop is a cycle. Use 64-bit-safe values for costs and distances.

Target complexity: O(n+m) time and memory; reconstructing a route takes O(route length).

Test your work in GitHub

The repository already contains examples and tests. You can run them through GitHub Actions without installing Python for the checking tools.

1. On the template page, choose Use this template → Create a new repository. Work in your own copy. It may be public or private; a private copy must be accessible to your instructor.

2. In submission.json, set language to Java, Python, Cpp or CSharp. For C++, use Cpp; for C#, use CSharp. Complete the Algorithms file for that language. You can work locally in your IDE, then commit and push your changes.

3. Open Actions → Public checks in your repository. A run starts after a push. If GitHub asks, enable Actions. Open the latest run for your commit and read the steps. Compile and launch checks that your program starts; Algorithm examples checks the algorithms, including edge cases and repeated requests. Red failures are expected while TODOs remain. Fix the first relevant error and push again.

4. Add your own examples in tests/student/ and record the input, expected result and actual result. Complete ANALYSIS.md: explain each algorithm in 3–5 sentences, include the Kruskal/Prim comparison and a route reconstruction example, and acknowledge assistance under the syllabus policy.

5. Before submitting, check that Public checks is green for the exact commit you will submit. These are the ready-made preparation tests. After the deadline, the instructor runs a separate, deeper check against that submitted commit and reviews the required methods. Passing the public tests does not guarantee that every possible case is correct.

If you want to run the checks locally, TESTING.md explains the optional launcher. The Python launcher is a checking tool for all four languages; it does not mean that your solution must be Python. CONTRACTS.md contains the exact JSON fields and full input bounds when you need a technical reference. Keep one JSON answer per request and reset state between requests; debug output must not appear in the program's JSON output.

What to submit

In Moodle's Online text, paste **two links**:

1. The exact commit containing your completed code, analysis and tests: https://github.com/YOUR_ACCOUNT/YOUR_REPOSITORY/commit/FULL_40_CHARACTER_SHA . Open your commit in GitHub and copy its URL; the full SHA fixes the version to be checked.

2. The successful Public checks run for that same commit: https://github.com/YOUR_ACCOUNT/YOUR_REPOSITORY/actions/runs/RUN_ID . Open the green run and copy its URL. Its commit must match the first link.

Save and click Submit assignment; confirm that Moodle shows Submitted for grading. Do not upload the whole project, enter a student ID or submit only a changing main-branch link. The commit identifies the work; the Actions link shows the completed public checks. Moodle records when you submitted. If you revise before submissions close, submit the new commit and its new test-run link using the available controls; if locked, contact the instructor. Later changes to your repository do not replace the commit you submitted.